

Vulnerabilidades em Dependências Transitivas em Projetos Go Populares no GitHub

TI6 · 2026.1 · Relatório Final

Gabriel Borges · Rafael Pierre · Lucas Barbosa · Rodrigo Carvalho · Gustavo Ceolin · Yago Vaz ·
Guilherme Costa

Geração automatizada · 29/04/2026 · base govulncheck atualizada em 2026-04-21

740 OSVs no dep graph (de 2.100 detectados) amostra de 10 repositórios	263 OSVs estritamente transitivas 35,5% das in-graph
991 d Idade mediana das vulns ≈ 2,7 anos	82% % vulns > 1 ano sem fix ritmo de remediação extremamente lento

1. Resumo executivo

Sobre 10 repositórios Go com mais de 2.000 estrelas no GitHub, executamos o pipeline **govulncheck** → **cruzamento OSV × árvore de deps** → **análise estatística**. Encontramos **740 OSVs** presentes nas árvores de dependência (35% de todos os 2.100 conhecidos pela base) e classificamos cada um quanto à origem (declarado em *go.mod* ou puxado pela cadeia transitiva).

Hipótese principal — confirmada com folga. Vulnerabilidades transitivas são prevalentes (densidade média 41,5%), e o tempo médio de exposição é alto: 82% das vulns têm mais de 1 ano desde a data de publicação. **H₀ rejeitada.**

Diferenças importantes vs ecossistemas comparados na literatura (npm, Maven):

- **Profundidade não prediz risco em Go** — o MVS (Minimum Version Selection) achata o grafo, promovendo módulos transitivos a *require // indirect* no *go.mod* do root.
- **Volume de transitivas prediz risco** — Spearman $\rho = +0,55$ entre *#transitivas* e *#true-trans-vulns*.
- **Iceberg universal de reachability** — apenas 8 OSVs (1,1% das in-graph) têm evidência de chamada/import via call graph. Bate com o ~1% de Mir et al. (Maven, SANER 2023).

2. Amostra e seleção

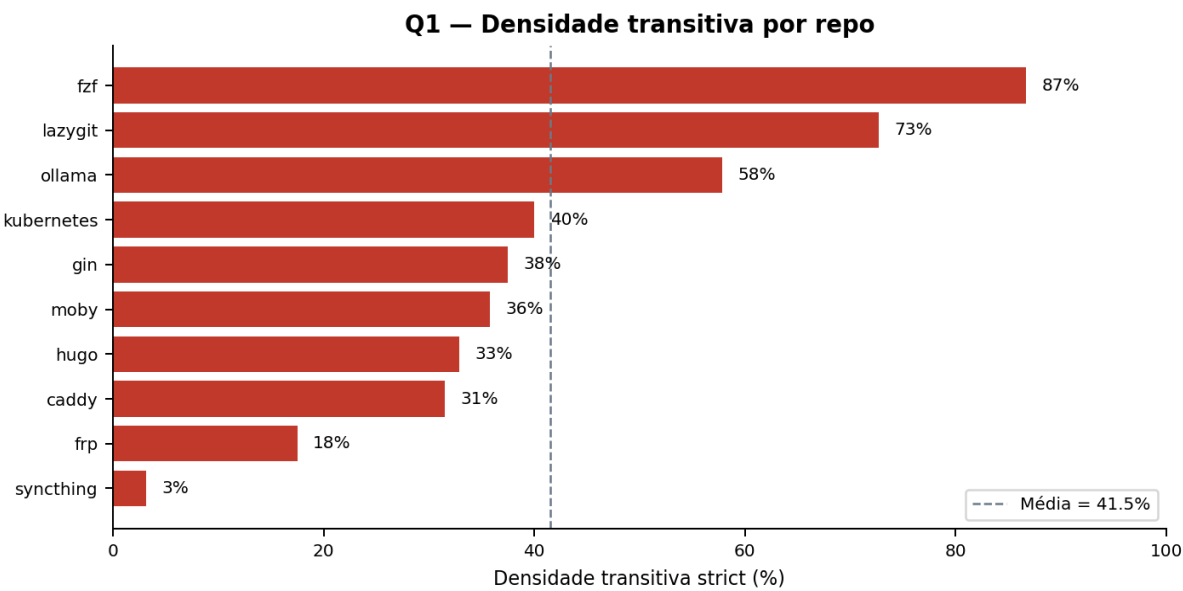
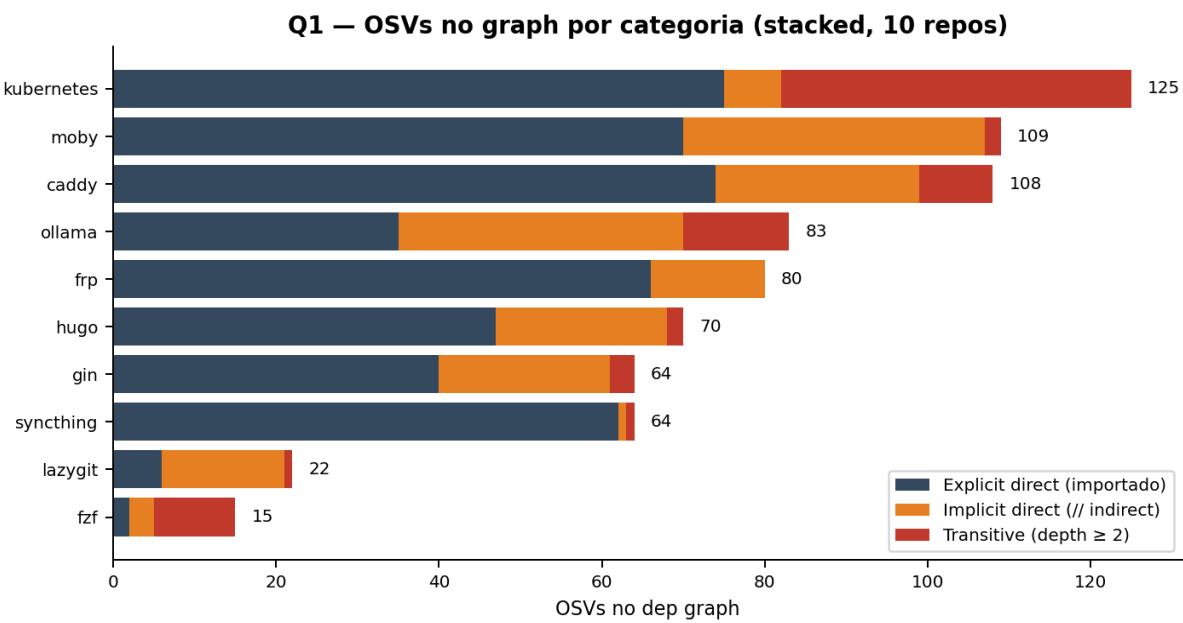
Query base: stars:>2000 language:Go sort:stars-desc. Aplicamos filtros operacionais: não-arquivado, não-fork, possui go.mod no root, pushedAt < 180 dias, fora da lista de exclusão (golang/go é toolchain; avelino/awesome-go é curated list). Auditoria completa em *data/repos-rejected.json*.

#	Repositório	■	Categoria
1	ollama/ollama	170.291	Runtime LLM
2	kubernetes/kubernetes	121.992	Orquestração
3	fatedier/frp	106.172	Reverse proxy
4	gin-gonic/gin	88.401	Framework HTTP
5	gohugoio/hugo	87.819	SSG
6	syncthing/syncthing	83.284	Sync de arquivos
7	junegunn/fzf	79.883	Fuzzy finder
8	jesseduffield/lazygit	77.214	TUI git
9	caddyserver/caddy	71.924	Web server
10	moby/moby	71.513	Engine Docker

3. Q1 — Frequência e densidade transitiva

Para cada OSV detectado, classificamos contra a árvore de deps do repo:

Categoria	Definição operacional
explicit_direct	require em go.mod sem // indirect — importado no source
implicit_direct	require em go.mod com // indirect — puxado pelo MVS, não importado
transitive	depth ≥ 2 no go mod graph
true_transitive	implicit_direct + transitive (não declarado pelo dev)



3.1 Tabela detalhada por repo

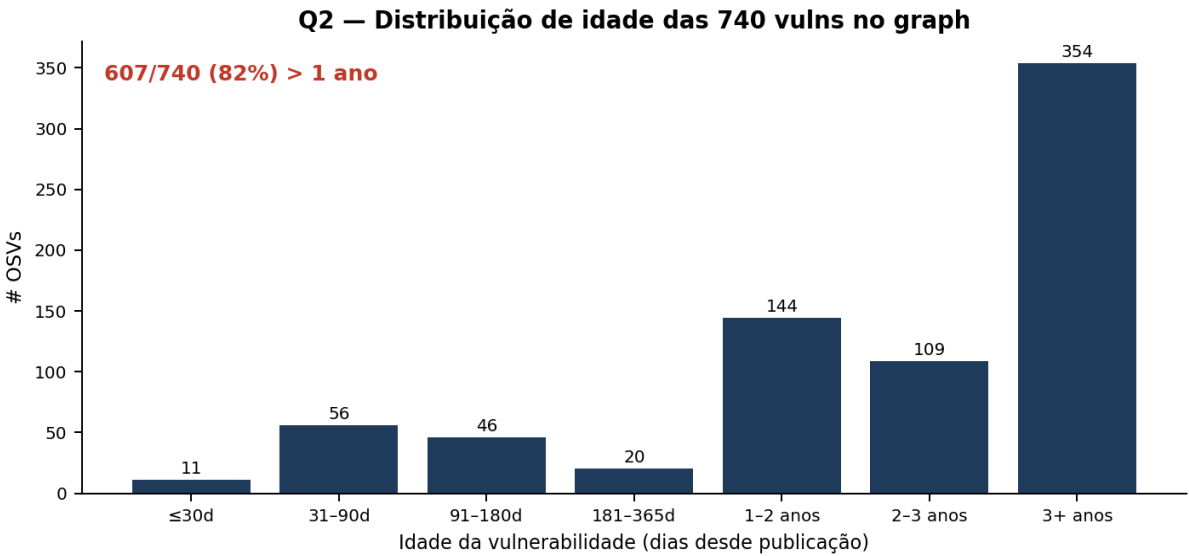
Repo	Total OSVs	inGraph	Expl	Impl	Trans (≥ 2)	True Trans	Density
fzf	151	15	2	3	10	13	87%
lazygit	158	22	6	15	1	16	73%
ollama	219	83	35	35	13	48	58%
kubernetes	261	125	75	7	43	50	40%
gin	200	64	40	21	3	24	38%
moby	245	109	70	37	2	39	36%
hugo	206	70	47	21	2	23	33%
caddy	244	108	74	25	9	34	31%
frp	216	80	66	14	0	14	18%
syncthing	200	64	62	1	1	2	3%

Observações. *syncthing* é outlier defensivo: 62 explicit + apenas 1 implicit no go.mod, strict density de 3%. Modelo de supply-chain hygiene exemplar. *junegunn/fzf* é o oposto: minimalista (12 deps no go.mod) mas 87% das vulns presentes vêm da cadeia transitiva — o minimalismo declarativo não protege da cadeia subjacente.

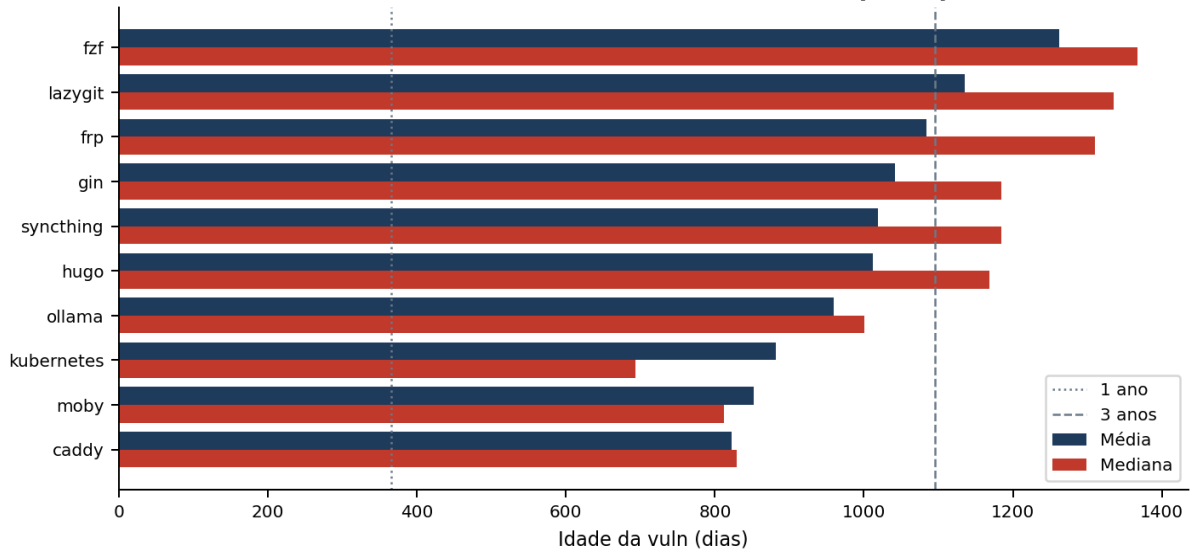
4. Q2 — Agilidade na remediação

Como o govulncheck reporta apenas vulns **atualmente presentes** (todas ainda não-fixadas), MTTR proper é incomputável sem scans históricos. Reportamos **vuln-debt-age** = dias desde a publicação da OSV (mínimo MTTR caso fixado hoje) e **tag inertia** = tags do projeto criadas após a publicação da OSV.

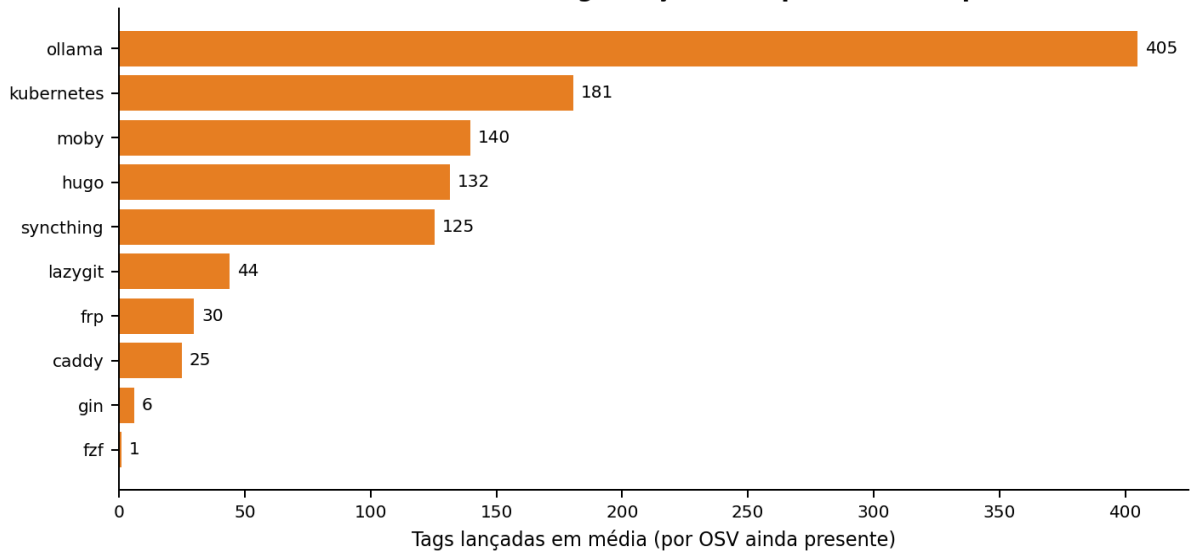
991 d Idade mediana ≈ 2,7 anos	953 d Idade média ≈ 2,6 anos
91% > 90 dias sem fix 673 de 740 OSVs	82% > 365 dias sem fix 607 de 740 OSVs



Q2 — Idade média e mediana das vulns por repo



Q2 — Inércia de versão: tags lançadas enquanto a vuln persistiu



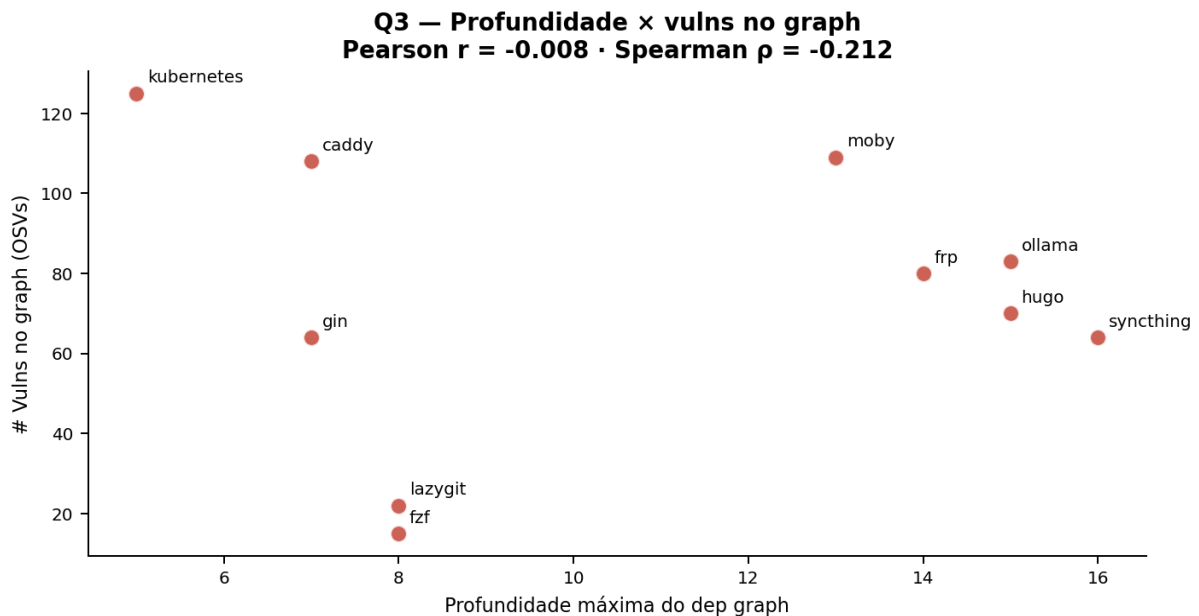
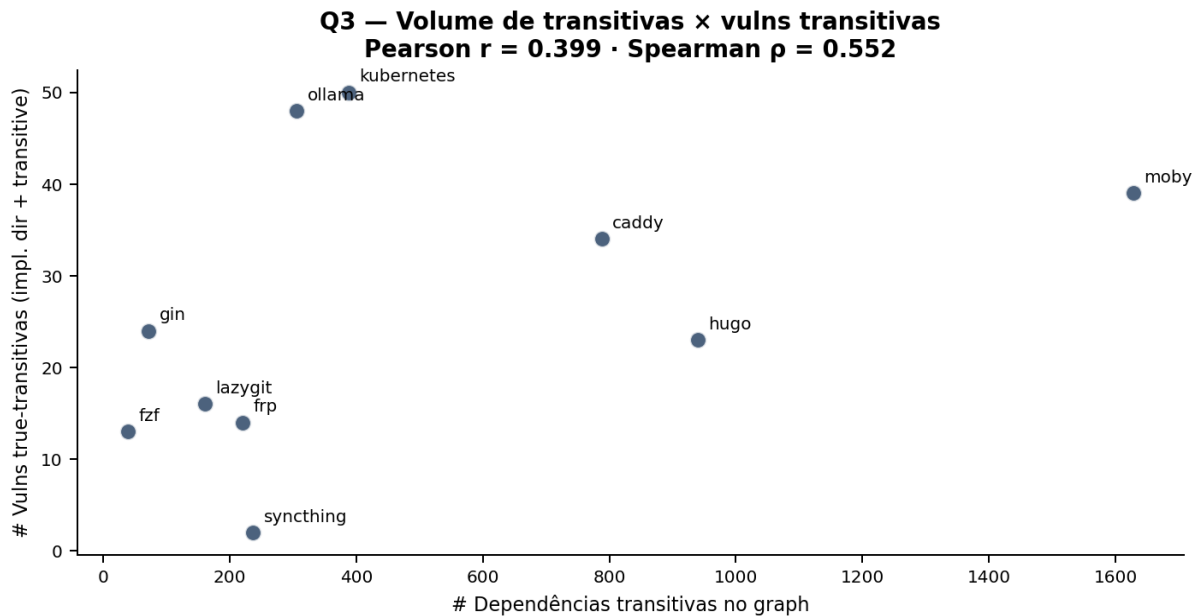
4.1 Tabela detalhada por repo (Q2)

Repo	Vulns	Idade média (d)	Idade mediana (d)	Idade max (d)	Tags média	Tags max
fzf	15	1262	1367	1841	1	1
lazygit	22	1135	1336	1841	44	69
frp	80	1084	1310	1841	30	49
gin	64	1042	1185	1841	6	14
syncthing	64	1019	1185	1841	125	219
hugo	70	1012	1168	1841	132	222
ollama	83	959	1001	1841	405	514
kubernetes	125	882	693	1841	181	444
moby	109	852	812	1841	140	222
caddy	108	822	830	1841	25	57

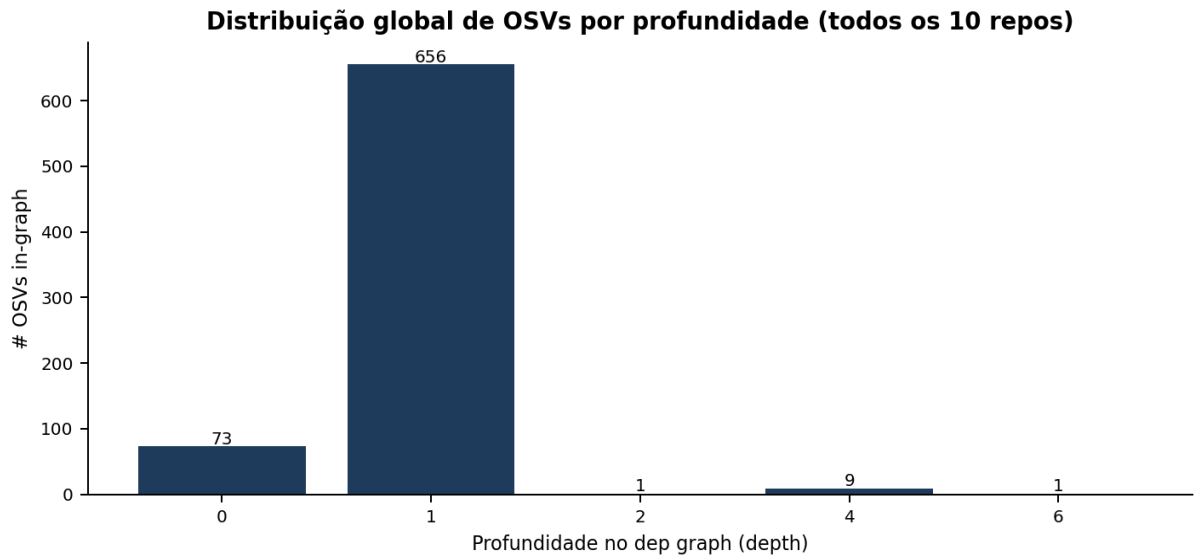
5. Q3 — Correlação superfície × vulns

Testamos duas hipóteses operacionais via Pearson e Spearman sobre os 10 pontos:

Hipótese	Pearson r	Spearman ρ	Interpretação
maxDepth × vulnsInGraph	-0.008	-0.212	Sem correlação
transitiveDeps × trueTransVulns	0.399	0.552	Moderada positiva

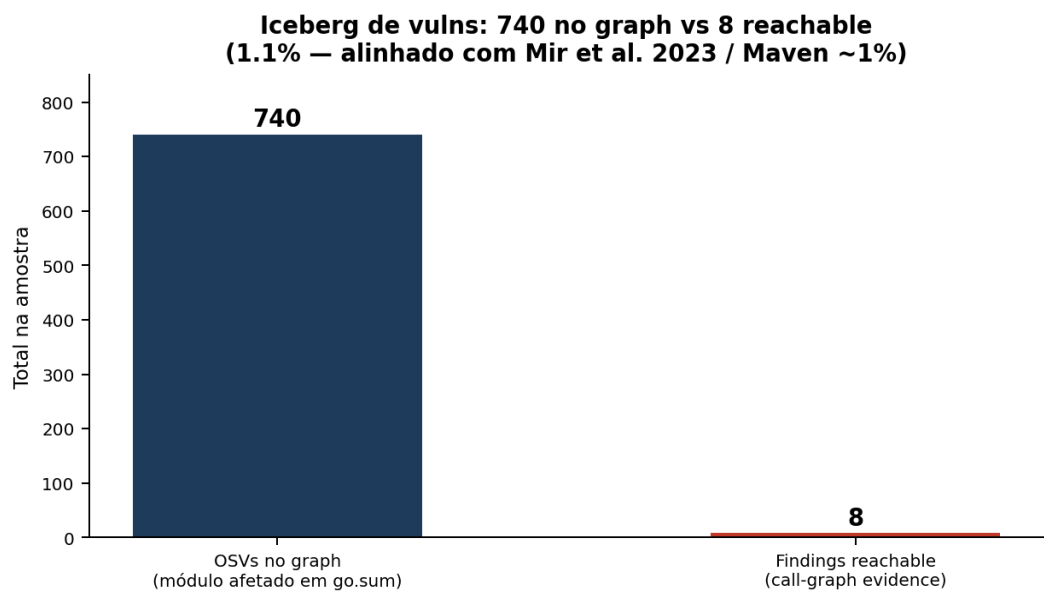


5.1 Distribuição global por profundidade



Insight. Em Go, o MVS achata a árvore: módulos transitivos são promovidos para *require // indirect* no go.mod do root, ficando em depth=1. Por isso kubernetes/k8s tem maxDepth=5 com 388 deps transitivas. Estudos de profundidade (Decan/Mens/Grosjean para npm) não se aplicam diretamente. Para Go, **volume é o preditor real, não profundidade.**

6. Findings com call-graph reachability



Apenas 8 OSVs únicos têm evidência de chamada/import — **1,1% das 740 in-graph**. Confirma o achado de Mir, Keshani & Proksch (SANER 2023) para Maven (~1%). O "iceberg" do supply chain é universal entre ecossistemas.

Repo	OSV	Module	Level	Fixed
frp	GO-2026-4479	github.com/pion/dtls/v3	imported	v3.0.11
frp	GO-2025-4233	github.com/quic-go/quic-go	required	v0.57.0
ollama	GO-2026-4514	github.com/buger/jsonparser	required	v1.1.2
ollama	GO-2026-4815	golang.org/x/image	required	v0.38.0
ollama	GO-2026-4961	golang.org/x/image	required	v0.39.0
ollama	GO-2026-4962	golang.org/x/image	required	v0.39.0
ollama	GO-2025-4134	golang.org/x/crypto	required	v0.45.0
ollama	GO-2025-4135	golang.org/x/crypto	required	v0.45.0

7. Conclusões

7.1 Resposta às hipóteses

Hipótese principal: «Projetos Go populares apresentam vulnerabilidades relevantes em deps transitivas, com tempo de correção maior que diretas». **Confirmada** sobre a amostra: densidade transitiva strict média de 41,5% e 82% das vulns > 1 ano sem fix.

H■: «Vulns transitivas são raras e não representam risco real». **Rejeitada** — 263 vulns true-transitivas estão simultaneamente presentes em 10 repos populares.

7.2 Implicações

- **Auditoria por volume, não por profundidade** em Go (MVS achata o grafo).
- **Vigilância sobre // indirect** em go.mod — fonte não negligenciável de risco. syncthing mostra que é possível minimizá-lo (3% strict density vs 87% no fzf).
- **govulncheck -scan symbol como padrão**, com fallback para -scan module quando código gerado quebra a compilação. A discrepância entre módulos com vuln (740) e símbolos chamáveis (8) é informação valiosa.
- **Tag inertia média de 128** sugere que projetos lançam dezenas/centenas de versões sem priorizar dep updates — oportunidade para automação (Renovate, Dependabot, Snyk).

7.3 Limitações e ameaças à validade

- **N=10** é amostra pequena. Mitigada por filtros explícitos e pela escolha dos top-10.
- **Snapshot atual** dos repos (clones shallow). MTTR proper requer scans históricos; usamos vuln-debt-age como proxy.
- **2/10 escaneados em -scan module** (ollama, syncthing) por falha de compilação — OSVs comparáveis, mas sem call-graph reachability nesses dois.
- **Base govulncheck atualiza diariamente** — snapshot 2026-04-21 anotado.
- **Matching OSV-módulo por nome** (sem semver range pós-cruzamento) — govulncheck já filtra por range antes; nossa pós-classificação só refina por categoria.

Apêndice — Pipeline de execução

Pipeline reproduzível em 6 etapas (todos os scripts em *scripts/*):

Etapa			Script			Saída		
1. Coleta			fetch-repos.js			data/repos.json		
2. Clonagem			clone-repos.js			repos/<owner>__<name>/		
3. Mapeamento			build-dep-tree.js			data/deps/<repo>.json		
4. Escaneamento			scan-repos.js			data/scans/<repo>.{ndjson,json}		
5a. Análise Q1+Q3			analyze-scans.js			data/analysis.json		
5b. Análise Q2			analyze-q2.js			data/analysis-q2.json		
6. Exportação			export-csv.js			data/csv/*.csv		

Referências bibliográficas relevantes

- Kumar, S.H.B.I. et al. *A Comprehensive Study on the Impact of Vulnerable Dependencies on Open-Source Software*. ISSRE 2024. **(artigo-semente)**
- Mir, A.M.; Keshani, M.; Proksch, S. *On the Effect of Transitivity and Granularity on Vulnerability Propagation in Maven*. SANER 2023. **(artigo-semente)**
- Zimmermann, M. et al. *Small World with High Risks: a study of security threats in the npm ecosystem*. USENIX Security 2019.
- Decan, A.; Mens, T.; Grosjean, P. *An empirical comparison of dependency network evolution in seven software packaging ecosystems*. Empirical SE 2019.
- Ponta, S.E.; Plate, H.; Sabetta, A. *Detection, Assessment and Mitigation of Vulnerabilities in Open Source Dependencies*. Empirical SE 2020.